FastAPI and ML Integrated AI-Agent

Data is raw, unorganized facts, numbers, text, images, or measurements collected from observations. In the context of technology, data is the foundational material that applications, analytics, and algorithms process to extract insights, make decisions, or generate predictions.

Here is a breakdown of the core concepts in the data and machine learning pipeline.

---

1. Data Ingestion

Data ingestion is the process of importing and loading raw data from an external source (like a database, API, or file) into a system or programming environment where it can be analyzed.

Sample File (`housing_data.csv`):

```
csv
id,area_sqft,bedrooms,age_years,price
1,1500,3,10,300000
2,2200,4,,450000
3,1800,3,5,350000
4,2200,4,12,
5,1500,3,10,300000
```

Code Example (Python/Pandas):

```python
import pandas as pd

# Ingest data from a CSV file into a Pandas DataFrame
df = pd.read_csv('housing_data.csv')
print("Raw Data:")
print(df.head())
```

2. Data Cleansing

Real-world data is messy. Data cleansing (or cleaning) involves identifying and correcting errors, handling missing values, and removing duplicates to ensure the dataset is accurate and reliable.

Code Example:

```
# 1. Drop duplicate rows (e.g., ID 1 and 5 are identical except for ID)
df = df.drop_duplicates(subset=['area_sqft', 'bedrooms', 'age_years',
'price'])

# 2. Handle missing values (NaN)
# Fill missing 'age_years' with the median age
median_age = df['age_years'].median()
df['age_years'] = df['age_years'].fillna(median_age)

# Drop rows where the target variable ('price') is missing
df = df.dropna(subset=['price'])

print("\nCleaned Data:")
print(df)
```

3. Data Wrangling

Data wrangling (or munging) is the process of transforming and mapping data from its raw, cleaned format into a more appropriate structural format for downstream analysis. This includes filtering, reshaping, merging, and changing data types.

Code Example:

```
# Ensure data types are correct (e.g., age should be an integer)
df['age_years'] = df['age_years'].astype(int)

# Filter for houses larger than 1600 sqft
large_houses = df[df['area_sqft'] > 1600]

# Add a categorical column based on numerical conditions
import numpy as np
df['property_size'] = np.where(df['area_sqft'] > 2000, 'Large', 'Medium')
```

4. EDA, FE, and FS

- **EDA (Exploratory Data Analysis):** Analyzing datasets to summarize their main characteristics, often using visual methods, to find patterns, anomalies, or relationships.
- **FE (Feature Engineering):** Creating new, meaningful input variables (features) from the existing data to improve a machine learning model's performance.
- **FS (Feature Selection):** Selecting the most relevant features to use in model construction, dropping redundant or irrelevant ones to simplify the model and reduce overfitting.

Code Example:

```
import seaborn as sns
import matplotlib.pyplot as plt

# --- EDA (Exploratory Data Analysis) ---
# Visualize the relationship between area and price
sns.scatterplot(x='area_sqft', y='price', data=df)
plt.title('Area vs Price')
plt.show()

# --- FE (Feature Engineering) ---
# Create a new feature: Price per square foot
df['price_per_sqft'] = df['price'] / df['area_sqft']

# --- FS (Feature Selection) ---
# Select only the features we want to feed into our model
# We drop 'id' (irrelevant) and 'price_per_sqft' (derived directly from
the target)
selected_features = ['area_sqft', 'bedrooms', 'age_years']
X = df[selected_features] # Input features
y = df['price']           # Target variable
```

---

5. What is Machine Learning (ML)?

Machine Learning is a subset of Artificial Intelligence (AI) that focuses on building systems that learn from data. Instead of explicitly programming a computer to solve a problem step-by-step, you feed it data and allow algorithms to discover patterns and map inputs to outputs autonomously.

Types of Machine Learning

- **Supervised Learning:** The model is trained on labeled data (input-output pairs). It learns the mapping to predict outputs for new, unseen inputs. *(Examples: Predicting house prices, classifying spam emails).*
- **Unsupervised Learning:** The model is given unlabeled data and must find hidden structures, patterns, or groupings on its own. *(Examples: Customer segmentation, anomaly detection).*
- **Reinforcement Learning:** An agent learns to make decisions by performing actions in an environment to maximize a reward. *(Examples: Chess-playing AI, self-driving car navigation).*

---

6. Linear Regression (In Detail)

Linear Regression is a fundamental **Supervised Learning** algorithm used for predicting a continuous numerical value (regression). It assumes a linear relationship between the input variables (features, X) and the single output variable (target, y).

The Math

For a simple linear regression (one feature), the model tries to fit a line to the data using the equation:

$$y = mx + b$$

Where:

- y is the predicted output.
- x is the input feature.
- m is the slope (weight), determining how much y changes for a unit change in x.
- b is the y-intercept (bias), determining where the line crosses the y-axis.

For **Multiple Linear Regression** (multiple features), the equation expands:

$$y = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$$

The algorithm uses an optimization method (like Ordinary Least Squares or Gradient Descent) to find the best weights (w) and bias (b) that minimize the **Mean Squared Error (MSE)**—the average squared distance between the predicted values and the actual data points.

Code Example (Scikit-Learn)

Here is how to implement a multiple linear regression model using the data prepared earlier.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Split data into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 2. Initialize the model
model = LinearRegression()

# 3. Train (fit) the model on the training data
# This is where the algorithm calculates the best weights (m) and bias (b)
model.fit(X_train, y_train)

# 4. Make predictions on the testing data
y_pred = model.predict(X_test)

# 5. Evaluate the model
```

```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Model Coefficients (Weights):", model.coef_)
print("Model Intercept (Bias):", model.intercept_)
print(f"Mean Squared Error: {mse}")
print(f"R-squared (Accuracy of fit): {r2}")

# Predict a brand new house
new_house = [[2000, 3, 2]] # 2000 sqft, 3 beds, 2 years old
predicted_price = model.predict(new_house)
print(f"Predicted Price for new house: ${predicted_price[0]:.2f}")
```

---

www.Vi4Academy.com

www.Vi4Academy.com